

Roteiro de Aula 1 – Otimização Não Linear

Disciplina: PCC179 – Otimização Não Linear

3 de outubro de 2025

1 Formulação Geral de um Problema de Otimização Não Linear

Um problema geral de otimização não linear pode ser formulado da seguinte forma:

$$\begin{aligned} \min_{x \in \mathbb{R}^n} \quad & f(x) \\ \text{sujeito a} \quad & g_i(x) \leq 0, \quad i = 1, \dots, m, \\ & h_j(x) = 0, \quad j = 1, \dots, p, \end{aligned}$$

onde:

- $x = (x_1, x_2, \dots, x_n)^\top \in \mathbb{R}^n$ é o **vetor de variáveis de decisão**;
- cada x_i representa uma variável individual;
- $f(x)$ é a **função objetivo**, possivelmente não linear;
- $g_i(x)$ representam as **restrições de desigualdade**;
- $h_j(x)$ representam as **restrições de igualdade**.

Interpretações de x em aplicações reais

- Em **finanças**: x pode representar os pesos de ativos em um portfólio.
- Em **engenharia**: x pode representar dimensões de uma estrutura (espessura, comprimento, etc.).
- Em **machine learning**: x pode representar parâmetros ou hiperparâmetros de um modelo.

2 Exemplos de Problemas Reais

Exemplo 1: Ajuste de Curvas (Regressão Não Linear)

Minimizar o erro quadrático entre uma curva não linear e dados experimentais:

$$\min_{\theta} \sum_{i=1}^N (y_i - f(x_i; \theta))^2$$

Exemplo 2: Portfólio de Investimentos (Markowitz)

O problema de otimização de portfólio de Markowitz busca alocar capital entre n ativos de forma a equilibrar **retorno esperado** e **risco**.

Formulação

- Vetor de decisão:

$$w = (w_1, w_2, \dots, w_n)^\top, \quad w_i \geq 0, \quad \sum_{i=1}^n w_i = 1$$

onde w_i é a fração do capital investida no ativo i .

- Retorno esperado do portfólio:

$$\mu_p = w^\top \mu$$

onde $\mu = (\mu_1, \dots, \mu_n)^\top$ são os retornos esperados dos ativos.

- Risco (variância do portfólio):

$$\sigma_p^2 = w^\top \Sigma w$$

onde Σ é a matriz de covariâncias dos retornos.

Problema de otimização

$$\begin{aligned} \min_{w \in \mathbb{R}^n} \quad & w^\top \Sigma w \\ \text{sujeito a} \quad & w^\top \mu \geq \mu_{\min}, \\ & \sum_{i=1}^n w_i = 1, \\ & w_i \geq 0, \quad i = 1, \dots, n. \end{aligned}$$

Interpretação

- O objetivo é minimizar o risco do portfólio.
- As restrições garantem: (i) retorno mínimo desejado, (ii) todo o capital investido, (iii) proibição de posições vendidas.
- O conjunto de soluções ótimas forma a **fronteira eficiente**.

Exemplo 3: Engenharia Estrutural – Dimensionamento de uma viga

Um engenheiro deseja projetar uma viga retangular de largura b e altura h para suportar um momento fletor máximo M . O objetivo é **minimizar a área da seção transversal** (e, portanto, o peso da viga), sujeita à restrição de que a tensão máxima não ultrapasse o limite admissível σ_{adm} .

$$\begin{aligned} \min_{b, h > 0} \quad & A(b, h) = b \cdot h \\ \text{sujeito a} \quad & \sigma(b, h) = \frac{M \cdot (h/2)}{I(b, h)} \leq \sigma_{adm}, \end{aligned}$$

onde $I(b, h) = \frac{bh^3}{12}$ é o momento de inércia da seção.

Substituindo:

$$\sigma(b, h) = \frac{6M}{bh^2}.$$

O problema final fica:

$$\begin{aligned} \min_{b, h > 0} \quad & b \cdot h \\ \text{sujeito a} \quad & \frac{6M}{bh^2} \leq \sigma_{adm}. \end{aligned}$$

3 scipy.optimize para otimização

3.1 Problemas de Otimização sem Restrições

Nesta seção vamos explorar métodos de otimização não linear para problemas **sem restrições**, usando a função `minimize` do `scipy.optimize`. Para isso, utilizaremos a **função de Rosenbrock** (banana function), um benchmark clássico em otimização.

1) Função de Exemplo

A função de Rosenbrock em duas variáveis é dada por:

$$f(x, y) = (1 - x)^2 + 100(y - x^2)^2$$

Ela possui um mínimo global em $(1, 1)$, onde $f(1, 1) = 0$. É conhecida por seu **vale estreito e curvo**, que desafia algoritmos de otimização.

```
import numpy as np
import matplotlib.pyplot as plt

def rosenbrock(x):
    return (1-x[0])**2 + 100*(x[1]-x[0]**2)**2

X, Y = np.meshgrid(np.linspace(-2, 2, 400),
                   np.linspace(-1, 3, 400))
Z = (1-X)**2 + 100*(Y-X**2)**2

plt.contour(X, Y, Z, levels=np.logspace(-0.5, 3.5, 20))
plt.scatter(1, 1, c='red', marker='x')
plt.title("Funcao de Rosenbrock (banana)")
plt.show()
```

2) Método Nelder-Mead

Método baseado em um **simplex**, não usa derivadas. Adequado para funções não suaves ou ruidosas, mas pode ser lento.

```
from scipy.optimize import minimize

x0 = [0, 0] # chute inicial
res = minimize(rosenbrock, x0, method='Nelder-Mead')
print(res)
```

3) Método BFGS (com gradiente)

Método quasi-Newton que atualiza uma aproximação da Hessiana usando o gradiente. Mais eficiente quando derivadas estão disponíveis.

```
grad = lambda x: np.array([
    -2*(1-x[0]) - 400*x[0]*(x[1]-x[0]**2),
    200*(x[1]-x[0]**2)
])

res = minimize(rosenbrock, x0, method='BFGS', jac=grad)
print(res)
```

4) Newton-CG (com Hessiana)

Método Newton-CG usa gradiente e Hessiana para acelerar a convergência, sendo indicado para funções suaves.

```
hess = lambda x: np.array([
    [2 - 400*(x[1]-x[0]**2) + 800*x[0]**2, -400*x[0]],
    [-400*x[0], 200]
])

res = minimize(rosenbrock, x0, method='Newton-CG',
               jac=grad, hess=hess)
print(res)
```

5) Newton-CG com Trust Region

Variação que utiliza **regiões de confiança**, mais robusta quando a Hessiana não é positiva definida.

```
res = minimize(rosenbrock, x0, method='trust-ncg',
               jac=grad, hess=hess)
print(res)
```

Resumo dos Métodos

- **Nelder-Mead**: não requer derivadas, útil em funções não suaves, mas mais lento.
- **BFGS**: método quasi-Newton eficiente quando gradiente está disponível.
- **Newton-CG**: usa gradiente + Hessiana, rápido em funções suaves.
- **Trust Region (Newton-CG)**: mais estável, adequado para casos em que a Hessiana pode ser indefinida.

Notas de documentação. Para detalhes completos, consulte:

- Nelder-Mead (método simplex, sem derivadas)¹
- BFGS (quasi-Newton, requer gradiente)²
- Newton-CG (usa gradiente e Hessiana, ou produto Hessiano-vetor)³
- trust-ncg (Newton conjugado dentro de regiões de confiança)⁴
- trust-krylov, trust-exact (variações do esquema de regiões de confiança)⁵

3.2 Problemas de Otimização com Restrições

Nesta seção seguimos com a função de Rosenbrock (banana) e adicionamos restrições:

$$f(x, y) = (1 - x)^2 + 100(y - x^2)^2, \quad \begin{cases} x + y \geq 1 & \text{(desigualdade)} \\ x - y = 0 & \text{(igualdade)} \end{cases}$$

¹<https://docs.scipy.org/doc/scipy/reference/optimize.minimize-neldermead.html>

²<https://docs.scipy.org/doc/scipy/reference/optimize.minimize-bfgs.html>

³<https://docs.scipy.org/doc/scipy/reference/optimize.minimize-newtoncg.html>

⁴<https://docs.scipy.org/doc/scipy/reference/optimize.minimize-trustncg.html>

⁵<https://docs.scipy.org/doc/scipy/reference/optimize.minimize-trustregion.html>

Método	Grad	Hess	Tipo	Conv.	Observações
Nelder-Mead	não	não	simplex (direto)	lenta	Bom p/ funções não suaves; robusto mas ineficiente em alta dim.
BFGS	sim	não	quasi-Newton	rápida	Usa aproximação da Hessiana; bom compromisso entre custo e convergência.
Newton-CG	sim	sim (ou Hv)	Newton conjugado	rápida	Usa grad e Hess; requer funções suaves; convergência quadrática.
trust-ncg	sim	sim (ou Hv)	região de confiança	robusta	Mais estável que Newton-CG puro; lida melhor com Hessianas indefinidas.

Tabela 1: Resumo de métodos sem restrição no `scipy.optimize.minimize`.

```
import numpy as np
from scipy.optimize import minimize
from scipy.optimize import LinearConstraint, NonlinearConstraint

# Objetivo (Rosenbrock)
def f(x):
    return (1-x[0])**2 + 100*(x[1]-x[0]**2)**2

# Gradiente e Hessiana (para metodos que aceitam)
def grad(x):
    return np.array([
        -2*(1-x[0]) - 400*x[0]*(x[1]-x[0]**2),
        200*(x[1]-x[0]**2)
    ])

def hess(x):
    return np.array([
        [2 - 400*(x[1]-x[0]**2) + 800*x[0]**2, -400*x[0]],
        [-400*x[0], 200]
    ])

# ----- Restricoes -----
# Linear: x + y >= 1 e x - y = 0
# Matriz A e limites [lb, ub] por linha
A = np.array([[1.0, 1.0], # x + y
              [1.0, -1.0]]) # x - y
lb = np.array([1.0, 0.0]) # [ >=1, =0 ]
ub = np.array([np.inf, 0.0]) # [ inf, 0 ]

lincon = LinearConstraint(A, lb, ub)

# Para SLSQP/COBYLA (dicionarios):
cons_slsqp = [
    {'type': 'ineq', 'fun': lambda z: z[0] + z[1] - 1.0}, # x+y-1 >= 0
    {'type': 'eq', 'fun': lambda z: z[0] - z[1]} # x - y = 0
]

# COBYLA aceita apenas INEQ: aproximamos x=y por duas ineqs
cons_cobyala = [
    {'type': 'ineq', 'fun': lambda z: z[0] + z[1] - 1.0}, # x+y-1 >= 0
    {'type': 'ineq', 'fun': lambda z: z[0] - z[1]}, # x - y >= 0
    {'type': 'ineq', 'fun': lambda z: z[1] - z[0]} # y - x >= 0
]

x0 = np.array([-1.2, 1.0]) # chute inicial classico
```

1) trust-constr (regiões de confiança com restrições)

Método robusto e geral para restrições não lineares e lineares. Aceita gradiente e Hessiana; usa regiões de confiança para passos estáveis.

```
res_tc = minimize(f, x0, method='trust-constr',
                 jac=grad, hess=hess,
                 constraints=[lincon],
                 options={'maxiter': 300, 'gtol': 1e-8, 'xtol': 1e-12})
print(res_tc.x, res_tc.fun, res_tc.success)
```

2) SLSQP (programação quadrática sequencial)

Aceita diretamente igualdade e desigualdade via dicionários. Usa gradiente se fornecido; costuma ser eficiente em problemas suaves.

```
res_slsqp = minimize(f, x0, method='SLSQP',
                    jac=grad, constraints=cons_slsqp,
                    options={'maxiter': 300, 'ftol': 1e-12})
print(res_slsqp.x, res_slsqp.fun, res_slsqp.success)
```

3) COBYLA (sem derivadas; só desigualdades)

Aproxima as restrições por aproximações lineares; útil quando derivadas não estão disponíveis. Igualdades precisam ser reescritas como duas desigualdades.

```
res_cobyla = minimize(f, x0, method='COBYLA',
                     constraints=cons_cobyla,
                     options={'maxiter': 1000, 'rhobeg': 0.5, 'tol': 1e-6})
print(res_cobyla.x, res_cobyla.fun, res_cobyla.success)
```

Resumo (restritos)

- **trust-constr**: interface com `LinearConstraint`/`NonlinearConstraint`; robusto; se beneficiar de grad/Hess.
- **SLSQP**: dicionários (eq/ineq); eficiente e prático para restrições suaves.
- **COBYLA**: dicionários, apenas ineq; não usa derivadas; bom quando grad/Hess não estão disponíveis.

Notas de documentação. Para detalhes completos, consulte:

- **trust-constr** (restrições via `LinearConstraint`/`NonlinearConstraint`)⁶
- **SLSQP** (restrições como dicionários eq/ineq)⁷
- **COBYLA** (apenas desigualdades, sem derivadas)⁸
- `LinearConstraint` / `NonlinearConstraint`^{9,10}

Método	Grad	Hess	Eq	Ineq	Formato restr.	Estrat./Observações
trust-constr	opcional	opcional	Sim	Sim	<code>Linear/NonlinearConstraint</code>	Regiões de confiança; robusto; bom p/ não lineares
SLSQP	opcional	não	Sim	Sim	dicionários eq/ineq	SQP; eficiente em suaves; sensível a mal condicionamento
COBYLA	não	não	Não (reformular)	Sim	dicionários ineq	Sem derivadas; lineariza restr.; pode exigir mais iterações

Tabela 2: Resumo de métodos com restrições no `scipy.optimize.minimize`.

⁶<https://docs.scipy.org/doc/scipy/reference/optimize.minimize-trustconstr.html>

⁷<https://docs.scipy.org/doc/scipy/reference/optimize.minimize-slsqp.html>

⁸<https://docs.scipy.org/doc/scipy/reference/optimize.minimize-cobyla.html>

⁹<https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.LinearConstraint.html>

¹⁰<https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.NonlinearConstraint.html>